

PDF Knowledge Assistant

Technical Retrospective — Conceptual Issues Encountered During Development

Project: Production-grade Retrieval-Augmented Generation system over PDF documents

Scope of this document

This retrospective documents **technical and architectural issues** encountered during construction of the PDF RAG Assistant. Each entry describes the observable symptom, the underlying technical cause, and the resolution applied. Issues are grouped by subsystem to support targeted review when working on similar components in future projects.

Table of Contents

1. Embedding subsystem: silent input truncation in the OpenAI embeddings API
2. Vector store: HNSW distance metric default conflicts with embedding-model output space
3. Vector store: Chroma `query()` response shape ambiguity
4. Vector store: deduplication strategy and the cost of re-embedding identical content
5. LCEL composition: routing query and context through parallel data paths
6. LCEL composition: type incompatibility between branches of `RunnableBranch`
7. Structured output: field-name divergence between schema and consumer code
8. FastAPI: dependency lifecycle and per-request initialization cost
9. SSE streaming: client-side buffering and the missing event-frame delimiter
10. Caching: scope-of-key invalidation across prompt changes
11. Evaluation: file-format assumptions in line-delimited data
12. Evaluation: misalignment between retrieval recall and answer faithfulness
13. Evaluation: Ragas internal dependency on an embeddings instance
14. Prompt engineering: regression of faithfulness following stricter instructions

1. Embedding Subsystem

Issues encountered while building the OpenAI embedding wrapper and integrating it with downstream chunking and retrieval logic.

1.1 Silent input truncation in the OpenAI embeddings API

Symptom. Embedding requests for long passages returned vectors without raising errors, but downstream retrieval quality on those passages was inexplicably poor compared to shorter passages from the same document.

Cause. The OpenAI embeddings endpoint silently truncates input that exceeds the model's token limit (8,191 tokens for `text-embedding-3-small`). The response is returned successfully with a partial-content embedding, producing a vector that represents only the prefix of the input. No warning or status field indicates that truncation occurred.

Resolution. Token-count validation was added as a hard precondition before any embedding API call. The `validate()` function counts tokens with `tiktoken` using the model-specific encoder and raises a `ValueError` for any input exceeding `MAX_TOKENS = 8191`. This converts a silent quality-degradation failure into a loud configuration error at ingestion time.

Generalization. Cloud APIs that silently truncate or downsample inputs are a recurring failure pattern. Defensive validation at the boundary (before the API call) is the senior pattern; relying on the API to surface input violations is not reliable.

1.2 Determinism assumptions on embedding output

Symptom. Two embedding calls for the same input string returned vectors that were nearly but not byte-identical. Cosine similarity between the two outputs measured at ≥ 0.9999 but was not exactly 1.

Cause. OpenAI embedding model inference involves GPU operations whose execution order is non-deterministic across server-side batches and load conditions. The model's behavior is deterministic in expectation but not at the bit level.

Resolution. Caching keys throughout the codebase use the **input string content hash**, never the output vector. Equality tests in eval code use similarity thresholds rather than exact vector comparison.

Generalization. Any caching or testing layer that depends on bit-level equality of LLM or embedding outputs is fragile. The contract is "outputs are similar within tolerance," not "outputs are identical."

2. Vector Store (ChromaDB)

Issues related to the Chroma persistent client, HNSW indexing, distance-metric configuration, and the API surface for querying collections.

2.1 HNSW distance metric default conflicts with OpenAI embedding output

Symptom. Initial retrieval quality on the gold set was lower than expected. Distance values returned from `collection.query()` were in an unexpected numeric range.

Cause. Chroma's default distance function is squared L2 (Euclidean). OpenAI embeddings are normalized to unit length and are designed for cosine similarity. While L2 distance on unit vectors is mathematically related to cosine distance, the ranking behavior under approximate nearest-neighbor search differs, and the absolute distance values are not interpretable in the expected range.

Resolution. Two changes were made:

- The collection is created with `metadata={"hnsw:space": "cosine"}` to override the default.
- An assertion immediately after collection creation verifies that the active metric is `cosine`. This guards against the failure case where Chroma silently retains a previously-set metric on a collection of the same name.

```
self.collection = self.client.get_or_create_collection(  
    name=collection_name,  
    metadata={"hnsw:space": self.DISTANCE_METRIC},  
)  
assert self.collection.metadata["hnsw:space"] == "cosine"
```

Generalization. Library defaults are chosen to minimize surprises across the broadest set of users, which means they are frequently wrong for any specific use case. Explicit configuration plus defensive verification of that configuration is the production pattern.

2.2 Chroma `query()` response shape ambiguity

Symptom. The first iteration of the retrieval-result-extraction code raised `IndexError` or returned malformed data when consumed by the chain.

Cause. Chroma's `collection.query()` returns nested lists at the top level of each field. The structure is `{"documents": [...], "metadatas": [...], "distances": [...]}` — a list of lists, where the outer list corresponds to the number of query vectors submitted. For a single query, the outer list has length one, and the actual results live at index `[0]`.

Resolution. The `VectorStore.query()` adapter unwraps the response once and returns a flat `list[dict]` shaped `[{"text": ..., "metadata": ..., "distance": ...}, ...]`. Callers never see the nested representation.

```
raw = self.collection.query(query_embeddings=query_embedding, n_results=k, ...)
documents = raw["documents"][0] # unwrap the outer list
metadatas = raw["metadatas"][0]
distances = raw["distances"][0]
```

Generalization. Wrapping third-party APIs in a thin adapter (Adapter pattern) confines representation quirks to a single file. Without the adapter, every consumer would need to reproduce the unwrap logic, multiplying the chance of error.

2.3 Deduplication strategy for repeat ingestion

Symptom. Re-running the CLI against an already-ingested PDF was producing redundant embedding API calls, despite the upsert behavior of the underlying Chroma collection.

Cause. Chroma's `upsert` deduplicates writes to the collection but does not prevent the application from paying the embedding cost. The embedding API call still occurred for chunks whose content was already stored; only the database write was suppressed.

Resolution. A `filter_new()` method was added to `VectorStore`. It computes the deterministic SHA-256 content hash for each candidate chunk, queries the collection to discover which IDs already exist, and returns only the subset of chunks that would be new. The ingestion pipeline embeds and adds only this subset:

```
def filter_new(self, texts):
    candidate_ids = [self._hash_id(t) for t in texts]
    existing = self.collection.get(ids=candidate_ids)
    existing_set = set(existing["ids"])
    # walk + filter, return only new texts/indices/ids
    ...
```

Generalization. Caching at the wrong layer can be more expensive than no caching. The relevant question for any cache is: "what specific cost am I avoiding?" Database-level deduplication does not reduce API-level cost. Content-hash IDs operating *before* the embedding call do.

3. LCEL and Chain Composition

Issues encountered while composing the retrieval-and-generation pipeline using LangChain Expression Language primitives.

3.1 Routing question and context through parallel paths

Symptom. Initial chain implementations lost access to the user's original question by the time the prompt template was rendered. The retrieval step consumed the question to produce context, but no downstream stage retained the question for inclusion in the prompt.

Cause. An LCEL chain passes a single value forward at each stage. A chain of the form `retriever | prompt | llm` forwards only the retriever's output (the formatted context); the original question is discarded after the retriever consumes it. The prompt template, however, has placeholders for both `{context}` and `{question}`, requiring both values to be present at the prompt stage.

Resolution. A `RunnableParallel` was introduced at the entry point to compute both values from the same input string, producing a dictionary that the prompt template can consume directly:

```
RunnableParallel(  
    context=RunnableLambda(retrieve_and_format),  
    question=RunnablePassthrough(),  
)
```

`RunnablePassthrough()` is the named primitive for "forward input unchanged." `RunnableLambda` wraps an arbitrary function as a `Runnable`. The output of the parallel stage is a dict with keys matching the prompt template placeholders.

Generalization. Declarative composition (the `|` operator) makes the data flow visible. The price is that data flowing through must be explicitly preserved when it needs to survive multiple stages. `RunnableParallel` + `RunnablePassthrough` is the standard idiom for this fan-out pattern.

3.2 Type incompatibility between branches of `RunnableBranch`

Symptom. When converting from the streaming text chain to the structured-output chain, the empty-context short-circuit branch returned a plain string while the main branch returned a Pydantic `Answer` instance. Downstream consumers expecting `Answer.text` attribute access failed when the empty branch fired.

Cause. `RunnableBranch` does not enforce type homogeneity across its branches at definition time. Both branches will execute and pass their output to whatever consumes the chain's result, regardless of whether the output types match.

Resolution. The empty branch was rewritten to return a valid `Answer` instance with all four required fields populated:

```
empty_branch = RunnableLambda(lambda _: Answer(  
    text=EMPTY_CONTEXT_MESSAGE,  
    citations=[],  
    confidence="low",  
    used_context=False,  
))
```

Generalization. Type consistency across conditional branches is a static-typing concern that Python's runtime does not enforce. When using `RunnableBranch`, treat the output type as an implicit contract: every branch must produce a value compatible with the rest of the pipeline.

4. Structured Outputs and Pydantic

Issues involving Pydantic schema design, `with_structured_output()` behavior, and field-name consistency between schema and consumers.

4.1 Field-name divergence between schema and consumer code

Symptom. The eval runner populated every result row with `actual_answer: null`. The runner reported zero errors and 30/30 successful invocations. The bug was only discovered after manually inspecting individual result records.

Cause. The `Answer` schema defined the answer text field as `text: str`. The eval runner extracted it with `payload.get("answer")`, accessing a key that did not exist in the serialized output. The `.get()` default returned `None` silently, and the runner stored `None` in every result without raising.

Resolution. All access to required schema fields was switched from `.get()` with implicit `None` defaults to bracket access (`payload["text"]`) for required fields. Required keys now raise `KeyError` if they are missing or misspelled, surfacing the bug immediately.

Generalization. Defensive `dict.get()` with implicit defaults is a useful pattern for genuinely optional fields, but it is harmful for fields that must exist for the program to be correct. The distinction is whether a missing value represents a tolerable absence or a logic error. Required fields should fail loud.

4.2 Streaming incompatibility with structured output

Symptom. Attempting to wrap the structured chain in an SSE streaming endpoint produced unusable output: either no tokens or a single dump of the entire JSON at once.

Cause. Token-level streaming requires that each partial output be independently meaningful (a string fragment is). Structured JSON output cannot stream because a partial JSON document is not valid JSON until complete. `with_structured_output(Answer)` internally accumulates the full LLM response before parsing it as JSON and constructing the Pydantic instance.

Resolution. Two endpoints were introduced with different output contracts:

- `/query` returns SSE-streamed plain text, suitable for interactive UIs.
- `/structured-query` returns a single JSON response containing the validated `Answer` object, suitable for programmatic clients.

Generalization. Streaming and structured-output are orthogonal feature sets in current LLM pipelines, and they are not jointly achievable for unbounded schemas. Designing the API around two distinct endpoints with different contracts is cleaner than trying to unify them.

5. FastAPI and Server Lifecycle

Issues related to dependency initialization cost, per-request overhead, and Server-Sent Events streaming.

5.1 Per-request initialization cost of the vector store

Symptom. Early API benchmarks showed several hundred milliseconds of overhead on every request, before any retrieval or LLM call had begun.

Cause. Constructing a `VectorStore` instance involves opening the SQLite-backed Chroma database, initializing a `PersistentClient`, and loading HNSW index metadata from disk into memory. This is acceptable as a one-time startup cost but unacceptable as a per-request cost.

Resolution. A process-level dictionary caches `VectorStore` instances by `document_id`. The `get_store()` helper performs lazy initialization on first access and returns the cached instance on subsequent calls.

```
_stores: dict[str, VectorStore] = {}

def get_store(document_id: str) -> VectorStore:
    if document_id not in _stores:
        _stores[document_id] = VectorStore(
            collection_name=f"pdf_{document_id}",
            persist_path=PERSIST_PATH,
        )
    return _stores[document_id]
```

Trade-off. The cache has no eviction policy. Under heavy multi-document load, the cache grows unboundedly. This is documented as future work; production deployment would need LRU eviction or a TTL.

5.2 Server-Sent Events delimiter and client-side buffering

Symptom. Initial SSE implementation produced no visible streaming on the client side. The Streamlit UI displayed the full answer at once after several seconds of apparent inactivity.

Cause. Two issues compounded:

- **Missing frame delimiter.** The SSE protocol requires each event to terminate with a double newline (`\n\n`). Without it, intermediate proxies and browsers buffer the response, releasing the entire stream to the client only at connection close.
- **Client buffering.** Using `httpx.post()` on the client reads the full response body before returning, even if the server is genuinely streaming. To preserve streaming behavior end-to-end, the client must use `httpx.stream()` and read events as they arrive.

Resolution.

```
# Server (FastAPI):
async def event_stream():
    async for token in chain.astream(question):
        yield f"data: {json.dumps({'token': token})}\n\n" # double newline
    yield f"data: {json.dumps({'done': True})}\n\n"

return StreamingResponse(event_stream(), media_type="text/event-stream")

# Client (Streamlit):
with httpx.stream("POST", url, json=body) as response:
    for raw_line in response.iter_lines():
        # parse and yield tokens incrementally
    ...
```

Generalization. Streaming requires end-to-end discipline. The protocol delimiter, the server's framing logic, and the client's read pattern must all be correct. A single buffer at any layer collapses the stream into a single response.

6. Caching Strategy

Issues involving cache scope, key invalidation, and the interaction of multiple cache layers with measured improvements.

6.1 Cache-key dependency on the full prompt text

Symptom. After modifying the system prompt to test a hypothesis, an eval re-run completed in 13 seconds (versus a baseline of 116 seconds), which was physically inconsistent with making real LLM API calls. Investigation revealed that the prompt change had not actually propagated to the active chain configuration, and the cache was returning previously-stored responses.

Cause. LangChain's `SQLiteCache` keys responses on the full prompt string plus the LLM configuration (model, temperature, etc.). If the prompt is unchanged, even unintentionally, the cache hits. The cost-saving behavior also obscures whether a code change is actually being exercised.

Resolution. Two practices were adopted:

- **Latency as a smoke test.** Eval runs that complete in implausibly short time are treated as suspicious by default. The expected latency floor for 30 real LLM calls is ~2 minutes.
- **Explicit cache invalidation** before measuring the impact of a change. Removing `.llm_cache.sqlite` before re-running guarantees that every measurement reflects real inference.

Generalization. Caches optimize for cost but obscure cause-and-effect. Any measurement of "system improvement" must verify that the changed component is actually being exercised in the measured run. Stage-level timing (separate `retrieve` and `llm_call` timers) makes this verifiable at a glance.

6.2 Cost-per-call asymmetry between LLM and embedding caches

Observation. Two cache layers were implemented: an LLM response cache (SQLite-backed, prompt-keyed) and an embedding cache (content-hash, ingestion-time). The two protect against different costs at different rates.

Quantification.

- LLM output (gpt-4o-mini): ~\$0.60 per 1M tokens
- Embeddings (text-embedding-3-small): ~\$0.02 per 1M tokens
- Per-token cost ratio: ~30×

Access patterns. In production traffic, each PDF is ingested approximately once but frequently-asked questions repeat across users (the FAQ pattern). The LLM cache therefore fires far more often than the embedding cache and avoids a 30× more expensive operation per hit. The LLM cache is the dominant cost saver in nearly all production workloads.

Generalization. When designing caching, two levers determine the saving: per-call cost of the cached operation, and access frequency of the cache. Optimizing the wrong layer (e.g., aggressive embedding caching when LLM calls dominate) yields negligible savings.

7. Evaluation Infrastructure

Issues encountered while building the gold-set runner, scoring logic, and Ragas integration.

7.1 Format mismatch in line-delimited gold-set file

Symptom. The scoring script crashed on the first line of the gold set with `JSONDecodeError: Expecting value: line 1 column 2 (char 1)`. The eval runner, however, had successfully iterated over the same file moments earlier.

Cause. The file extension was `.jsonl` but the content was a JSON array — a single top-level list containing 30 objects, formatted with line breaks for readability. The eval runner's loader was permissive enough to tolerate this in one form; the scorer's loader was strict and parsed each line individually, failing on the bare `[` character.

Resolution. The file was converted in place from JSON-array format to true JSONL (one object per line, no surrounding brackets):

```
data = json.loads(open('evals/gold_set.jsonl').read())
with open('evals/gold_set.jsonl', 'w') as f:
    for entry in data:
        f.write(json.dumps(entry) + '\n')
```

Generalization. File extensions are conventions, not enforced contracts. When parsing fails unexpectedly, verify the actual byte structure of the file with `file`, `head`, and `wc -l` before assuming the format matches its name. JSONL specifically requires one independently-valid JSON object per line — no enclosing brackets, no inter-object commas.

7.2 Misalignment between retrieval recall and answer faithfulness

Observation. Baseline metrics on the 30-question gold set showed retrieval hit-rate of 80% but Ragas faithfulness of only 0.47. The gap represents a class of failure that retrieval-only metrics cannot detect.

Diagnosis. Retrieval hit-rate measures whether the correct chunk was retrieved. Faithfulness measures whether the generated answer's claims are actually supported by the retrieved chunks. A high retrieval rate with low faithfulness indicates that the LLM has prior knowledge of the source material (in this case the Transformer paper is in its training data) and is producing answers that draw on internal knowledge rather than the supplied context.

Architectural implication. A single metric is insufficient to characterize a RAG system. Minimum viable evaluation requires at least:

- **Retrieval recall** (or hit-rate): are the right chunks being fetched?
- **Faithfulness**: is the answer grounded in those chunks?
- **Answer relevancy**: does the answer address the question?
- **Context precision**: are the retrieved chunks all useful?

Generalization. Different metrics catch different failure modes. A system optimized only against retrieval will tolerate hallucination; a system optimized only against faithfulness will tolerate irrelevant retrievals. Production RAG observability requires a metric portfolio, not a single score.

7.3 Ragas `answer_relevancy` requires an embeddings instance

Symptom. The Ragas evaluation produced `nan` for `answer_relevancy` across all 30 questions, with stack traces referencing `AttributeError: 'OpenAIEmbeddings' object has no attribute 'embed_query'`.

Cause. `answer_relevancy` works by asking the LLM to generate paraphrased questions that the answer could have come from, embedding the paraphrases and the original question, and measuring cluster tightness. It therefore requires a working embeddings instance. Ragas attempts to auto-pick a default embeddings wrapper, but this default is incompatible with recent versions of the OpenAI/LangChain packages; the wrapper it constructed lacked the expected `embed_query` method.

Resolution. An explicit `embeddings=` parameter was passed to `evaluate()`:

```
scores = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision],
    embeddings=OpenAIEmbeddings(model="text-embedding-3-small"),
)
```

Generalization. When integrating with a framework that auto-configures dependencies, version skew between the framework and its dependencies is a recurring source of silent failure. Passing dependencies explicitly is more verbose but immune to this class of bug.

8. Prompt Engineering Experiment

A documented experiment on whether stricter grounding instructions in the system prompt would improve faithfulness scores.

8.1 Faithfulness regression following stricter prompt instructions

Hypothesis. A revised system prompt containing explicit anti-hallucination clauses ("Never use outside knowledge," "Do not paraphrase beyond what the context supports," and a fixed-string abstain instruction) would increase faithfulness by reducing the LLM's reliance on training-data knowledge of the source paper.

Result. The hypothesis was falsified by measurement.

Metric	v1 (baseline)	v2 (stricter)	Δ
Retrieval hit-rate	80.0%	86.7%	+6.7%
Faithfulness	0.47	0.42	-0.05
Answer relevancy	0.82	0.79	-0.03
Context precision	0.68	0.78	+0.10

Interpretation. Stricter abstain instructions cause the model to hedge more frequently and produce vaguer prose. Ragas's faithfulness metric measures the proportion of claims in the answer that are supported by the retrieved context; vaguer answers with fewer concrete claims can score lower because each claim is less directly attributable to a specific context span. The improvement in context precision is uncorrelated with the prompt change (the retriever was unchanged) and is attributed to HNSW ranking variance between runs.

Conclusion. Prompt changes are experiments, not fixes. The intuition "stricter instructions = better grounding" is plausible but not validated; the actual behavior depends on the model's response to instruction density and on the specific metric being optimized. The experiment is documented as a result rather than reverted, because the evaluation harness — not the prompt — is the artifact worth preserving.

Generalization. The gap between "what should work in theory" and "what measurably works" is wide in LLM systems. Eval infrastructure is what closes the gap. Without it, a plausible-sounding prompt change ships as an improvement when in fact it is a regression.

Closing Note

The issues catalogued above span four broad areas: **API contract verification** (silent truncation, default mismatches, response shape), **system composition** (LCEL data flow, type compatibility, streaming versus structured output), **caching mechanics** (key scope, layer selection, invalidation discipline), and **evaluation rigor** (file format, metric portfolio, dependency wiring).

The recurring theme across all four is that the failures are not exotic. They are standard software engineering issues — boundary validation, configuration verification, format assumptions, measurement integrity — that happen to surface in an AI engineering context. The discipline that prevents them is the same discipline that prevents bugs in any system: read the API contract, configure explicitly, measure honestly, validate inputs at boundaries.

This document is intended as a personal reference for future projects of similar scope. Issues are listed in the order they were encountered to preserve the chronological dependency between them: earlier resolutions (e.g., the adapter pattern in §2.2) made later resolutions (e.g., the evaluation pipeline in §7) tractable.